espp/filters

1.0.28

Latest stable version: 1.1.7

Star 0

uploaded 9 months ago

Filter component for ESP-IDF

Readme

Dependencies 4

Dependents 1

Examples 1

Versions 49

Links

- Homepage
- Documentation
- Repository
- MIT

Supports all targets

Maintainer

- William Emfinger <waemfinger@gmail.com>

Tags

cpp Component Filter Butterworth Complementary Kalman Lowpass Madgwick SoS Transfer

To add this component to your project, run:

```
idf.py add-dependency "espp/filters^1.0.28"
```

download archive

Stats

Archive size ~ 19.11 KB

Downloaded in total 9.4k times

Weekly Downloads (All Versions)

283 Jul 27, 2026 — Aug 2, 2026



This version: 607 times

Badge



HTML | Markdown

report abuse

Readme

Filters Component



The `filters` component contains various types of filters that can be used for various signal processing and state estimation tasks.

Table of Contents

- Filters Component
  - Biquad Filter
  - Butterworth Filter
  - Complementary Filter
  - Kalman Filter
  - Lowpass Filter
  - Madgiwck Filter

- Simple Lowpass Filter
- SoS (Second-Order Sections) Filter
- Transfer Function
- Example

Biquad Filter

The `BiquadFilter` class provides an implementation of a Digital Biequad Filter with implementations for both the `Direct Form 1` and `Direct Form 2`.

Butterworth Filter

The `ButterworthFilter` class provides an implementation of a Digital Butterworth Filter, implemented as biquad sections (second order sections).

Complementary Filter

The `ComplementaryFilter` provides a simple implementation of a filter specifically designed to combine the outputs of a gyroscope and an accelerometer to produce a more accurate estimate of the orientation of an object. This filter is provided for completeness and simplicity, but the `KalmanFilter` or `MadgwickFilter` are generally preferred for this purpose.

Kalman Filter

The `KalmanFilter` class implements a Kalman filter for linear systems. The filter can be used to estimate the state of a linear system given noisy

measurements. The filter can be configured for an arbitrary number of states and measurements. You can also specify the process noise and measurement noise covariances.

To use the filter, you must first create an instance of the `KalmanFilter` class, then call the `predict` and `update` methods to estimate the state of the system. To get the current state estimate, call the `get_state` method.

Lowpass Filter

The `LowpassFilter` class provides an implementation of a digial lowpass infinite impulse response (IIR) filter, which leverages the hardware acceleration provided by esp-dsp and which leverages the vector instructions on the espressif processors.

Madgiwck Filter

The `MadgwickFilter` implements the Madgwick algorithm for orientation estimation using an IMU. The algorithm is based on the paper `An efficient orientation filter for inertial and inertial/magnetic sensor arrays` _ by Sebastian Madgwick. It supports state / orientation estimation for both 6-axis IMUs as well as for 9-axis IMUs.

Simple Lowpass Filter

The `SimpleLowpassFilter` class provides an implementation of a simple moving average filter with a configurable time constant.

SoS (Second-Order Sections) Filter

The `SosFilter` class provides an implementation of a Second Order Sections (SoS) filter. For more information please see series second-order sections as well as Digital Biquad Filter.

Transfer Function

The `TransferFunction` struct provides a simple container for storing the A and B coefficients for an N-th order transfer function.

Example

This example shows how to use a `LowpassFilter`, `ButterworthFilter`, and `SimpleLowpassFilter` from the `filters` component to filter signals. This example simply operates on random perturbations of auto-generated data.

© 2020-2026 Espressif Systems Cookie Policy Terms of use Privacy Policy Status